Unit 1: Introduction to Data Warehousing

1.1 Data Lifecycle

Collection → Storage → Usage → Archival → Destruction

- **Collection/Creation**: Data enters org via entry, acquisition, sensors
- **Storage**: Secure storage with backup and recovery
- **Usage**: View, process, modify, share data
- **Archival**: Remove from active env, keep copy for future use
- **Destruction**: Permanent removal of all copies

1.2 Types of Data

Type	Description	Example
Structured	Rows & columns, fixed schema, RDBMS/SQL	Relational tables
Semi-structured	Tags/markers, not rigid schema, XML	Email, XML
Unstructured	No data model, NoSQL/Data Lakes	Photos, video, audio, social media

1.3 Data Warehouse vs Operational Database (OLTP vs OLAP)

Feature	OLTP (Operational DB)	OLAP (Data Warehouse)
Purpose	Day-to-day transactions	Analysis & decision making
Data	Current data only	Historical data
Processing	Write-heavy (insert/update)	Read-heavy (aggregation)
Normalization	Normalized (less redundancy)	Denormalized (faster reads)
Model	ER model	Star/Snowflake schema
ACID	Strictly enforced	Less strict
Users	Data workers	Knowledge workers

1.4 Data Warehouse Key Features (SVIN)

- **S**ubject-Oriented: Organized around subjects (customer, product, sales)
- **T**ime-Variant: Maintains historical data over time
- **I**ntegrated: Consistent naming, encoding from multiple sources
- **N**on-Volatile: Data not deleted/updated; new versions inserted

1.5 ETL Process

Extract → Transform → Load

- **Extract**: Gather data from multiple source systems
- **Transform**: Clean, format, aggregate, convert
- **Load**: Insert into data warehouse (periodic refresh)

1.6 Multidimensional Data Model

- **Data Cube**: n-dimensional representation of data
- **Dimensions**: Entities for analysis (e.g., time, item, location)
- **Fact Table**: Contains numeric measures + foreign keys to dimension tables
- **Dimension Table**: Describes dimension attributes

1.7 Conceptual Schemas

Star Schema:

- Central fact table + denormalized dimension tables (star shape)
- Simpler queries, fewer joins, faster performance
- Higher redundancy

Snowflake Schema:

- Normalized dimension tables (split into sub-tables)
- Saves storage, more joins, slower queries

Fact Constellation (Galaxy Schema):

- Multiple fact tables sharing dimension tables
- Complex, supports multiple business processes

Example: Sales DW with dimensions: Time, Item, Location; Fact tables: Sales (units_sold, amount) and Shipping (dollars_cost, units_shipped)

1.8 OLAP Operations

Operation	Description
Roll-up (Aggregation)	Climb concept hierarchy / reduce dimensions
Drill-down	Reverse of roll-up — add detail
Slice	Select one dimension → sub-cube
Dice	Select ≥2 dimensions → sub-cube
Pivot (Rotate)	Re-orient cube view

Example: Sales cube with dimensions Time (Q1,Q2,Q3,Q4), City (KTM, PQR), Item (Laptop, Phone). Roll-up: Q1+Q2+Q3+Q4 → Yearly. Slice: Time=Q1 → 2D view. Dice: Time=Q1,Q2 AND City=KTM → sub-cube.

1.9 Need for Data Warehousing

- Decision makers need consolidated, historical data from multiple sources
- Operational databases are optimized for OLTP, not analysis
- Provides a single source of truth for reporting and BI
- Improves query performance — analytical queries don't slow down operational systems

1.10 Data Warehouse Implementation

- **Top-down approach** (Inmon): DW first, then data marts — centralized, consistent
- **Bottom-up approach** (Kimball): Data marts first, then DW — faster to deploy
- **ETL pipeline**: Extract from sources → Transform (clean, map, aggregate) → Load into DW
- **Data refresh**: Periodic (nightly, weekly) or near real-time
- **Key considerations**: Data quality, scalability, security, metadata management

1.11 Architecture of Data Warehouse (Three-Tier)

Bottom Tier — Data Source & Storage Layer:

- Data sources: Operational DBs, flat files, external data, ERP/CRM systems
- ETL Process: Extract → Transform (clean, map, aggregate) → Load into DW
- Central data warehouse database
- Data marts (departmental subsets)

Middle Tier — OLAP Server Layer:

- **ROLAP**: Extended RDBMS, maps multidimensional to relational operations
- **MOLAP**: Direct multidimensional storage & operations (faster)
- **HOLAP**: Hybrid — detailed data in RDBMS, aggregates in MOLAP

Top Tier — Front-end Layer:

- Query/reporting tools (BusinessObjects, PowerBI)
- Analysis/visualization tools (Tableau, Excel)
- Data mining tools

1.12 Types of Data Marts

Type	Description
Dependent	Built from existing DW — consistent with enterprise data
Independent	Built directly from operational sources — faster but isolated
Hybrid	Combines both — some data from DW, some directly from sources

1.13 Components of Data Warehouse

- **Data Mart**: Subset of DW for specific department/business line
- **Components**: Central DB, ETL tools, Metadata, Access tools
- **Metadata**: Data about data (technical + business)
- **Curse of Dimensionality**: Total cuboids = $\prod(L_i + 1)$

1.14 Data Warehouse Implementation Process

Source Systems → ETL → Staging Area → DW/DM → OLAP → Reporting/Analysis

- **Requirement analysis**: Understand business needs, identify KPIs
- **Data modeling**: Design star/snowflake schema, define dimensions and facts
- **ETL development**: Extract from sources, transform, load into DW
- **Build & test**: Populate DW, verify data integrity, optimize queries
- **Deploy & maintain**: Roll out to users, schedule refreshes, monitor performance

1.15 Trends in Data Warehousing

- Cloud DW (Snowflake, BigQuery, Redshift)
- Real-time data warehousing
- Data lakehouse architecture
- AI/ML integration

Unit 2: Introduction to Data Mining

2.1 Motivation for Data Mining

- **Explosive data growth**: Massive data from business, science, web — manual analysis impossible
- **Hidden patterns**: Trends, correlations, and anomalies not obvious to humans
- **Competitive advantage**: Data-driven decisions outperform intuition-based ones
- **Fraud and risk detection**: Identify fraudulent transactions, insurance claims, credit risks
- **Need for prediction**: Forecast sales, customer churn, market trends, disease outbreaks
- **Internet growth**: Web generates enormous user data — browsing, clicks, purchases, social media
- **Business intelligence**: Customer segmentation, market basket analysis, targeted marketing

2.2 Data Mining System

A data mining system is an integrated platform that discovers knowledge from large datasets.

Components:

- **Database/Data Warehouse Server**: Provides source data
- **Data Mining Engine**: Core component — executes mining algorithms (classification, clustering, association, etc.)
- **Pattern Evaluation Module**: Measures interestingness of discovered patterns using thresholds (support, confidence)
- **Knowledge Base**: Domain knowledge, concept hierarchies, constraints to guide mining
- **User Interface/GUI**: Visualization, interactive querying, report generation

Architecture Flow: Data Sources → DB/DW Server → Mining Engine → Pattern Evaluation → GUI (Knowledge Base guides all stages)

2.3 KDD Process (Knowledge Discovery in Databases)

Data → **Cleaning** → **Integration** → **Selection** → **Transformation** → **Mining** → **Pattern Evaluation** → **Presentation**

- Data mining is the essential step of KDD — extracting hidden patterns
- Architecture: Data Sources → DB/DW Server → Mining Engine → Pattern Eval → GUI

2.4 Data Mining Functionalities

Functionality	Description
Concept/Class Description	Characterization (summarize) + Discrimination (contrast)
Association	Find frequent itemsets / correlation rules
Classification	Predict categorical label (supervised)
Regression	Predict continuous value (supervised)
Clustering	Group similar objects (unsupervised)
Outlier Analysis	Detect unusual data points
Evolution Analysis	Model behavior over time (time-series)

2.5 Data Object & Attribute Types

Type	Description	Example
Nominal	Categories, no order	Hair color, marital status
Ordinal	Ordered categories	Size: S < M < L

Type	Description	Example
Interval	Equal intervals, no true zero	Temp °C
Ratio	True zero, ratios meaningful	Height, weight

2.6 Statistical Description of Data

Measures of Central Tendency:

- **Mean** (μ): Σx_i / N — sensitive to outliers
- **Median**: Middle value when sorted — robust to outliers
- **Mode**: Most frequent value — useful for categorical data
- **Trimmed Mean**: Remove top/bottom p%, then average — balances mean and median

Measures of Dispersion:

- **Range**: Max − Min
- **Variance** (σ^2): $\Sigma(x_i - \mu)^2$ / N
- **Standard Deviation** (σ): $\sqrt{\sigma^2}$ — spread in original units
- **Interquartile Range (IQR)**: Q3 − Q1 — robust to outliers

Example: Data {2, 4, 6, 8, 10}, N=5. Mean = 6. Variance = $[(2-6)^2+(4-6)^2+(6-6)^2+(8-6)^2+(10-6)^2]/5 = (16+4+0+4+16)/5 = 8$. SD = $\sqrt{8} \approx 2.83$

- **Trimmed Mean**: Remove top/bottom p%, then average

2.7 Data Mining Issues

- Mining methodology
- User interaction
- Efficiency/scalability
- Data diversity
- Security/social issues

2.8 Applications of Data Mining

- Market basket analysis, fraud detection, bioinformatics, web mining
- Customer segmentation, recommendation systems
- Medical diagnosis, financial forecasting

Unit 3: Data Preprocessing

Real-world data is dirty (incomplete, noisy, inconsistent). Preprocessing transforms raw data into a clean, consistent format suitable for mining. The preprocessing steps follow this order:

Step 1 — Data Cleaning → **Step 2 — Data Integration** → **Step 3 — Data Transformation** → **Step 4 — Data Reduction** → **Step 5 — Data Discretization**

3.1 Data Quality Problems

Problem	Description
Incomplete	Missing attribute values
Noisy	Errors or outliers
Inconsistent	Discrepancies in data

3.2 Step 1: Data Cleaning

Handling Missing Values:

- Ignore tuple
- Fill manually
- Global constant ("Unknown")
- Attribute mean / class-mean
- Most probable value (regression / Bayesian / DT)

Smoothing Noisy Data:

- **Binning**: Mean/median/boundary smoothing
- **Clustering**: Group similar, detect outliers
- **Regression**: Linear/non-linear smoothing

3.3 Binning Example (Cleaning)

Sorted: 4, 8, 15, 21, 21, 24, 25, 28, 34

Bin1: [4,8,15], Bin2: [21,21,24], Bin3: [25,28,34]

Method	Bin1	Bin2	Bin3
Mean	[9,9,9]	[22,22,22]	[29,29,29]
Median	[8,8,8]	[21,21,21]	[28,28,28]
Boundaries	[4,4,15]	[21,21,24]	[25,25,34]

3.4 Step 2: Data Integration

- Combine multiple sources → coherent data store
- **Entity Identification Problem**: Matching entities across sources
- **Redundancy Detection**: Correlation analysis
- Chi-square (nominal)
- Correlation coefficient (numeric)

3.5 Step 3: Data Transformation

Technique	Description
Smoothing	Remove noise

Technique	Description
Aggregation	Summarization
Discretization	Numeric → concepts
Normalization	Scale data to range
Attribute Construction	Create new attributes

Min-Max Normalization: $v' = (v - min) / (max - min) \times (new_{max} - new_{min}) + new_{min}$

Z-Score Normalization: $v' = (v - \mu) / \sigma$

Example: Values {10, 20, 30, 40, 50}. Min=10, Max=50. Min-max (range 0-1): 10→0, 30→0.5, 50→1. Z-score: μ=30, σ≈14.14. 10→−1.41, 30→0, 50→1.41

3.6 Step 4: Data Reduction

Approach	Methods
Dimensionality	PCA, Wavelet transforms, Attribute subset selection
Numerosity	Parametric (regression, log-linear), Nonparametric (histograms, clustering, sampling)
Compression	Lossless vs lossy

3.7 Step 5: Data Discretization & Concept Hierarchy

- **Discretization**: Binning, histogram, clustering, decision tree
- **Concept Hierarchy**: Replace low-level values with high-level concepts
- E.g., age: young / middle / senior

3.8 Data Mining Primitives (DMQL)

Primitives specify what to mine — defined using DMQL (Data Mining Query Language):

- **Task-relevant data**: Which DB, tables, attributes to mine
- **Kind of knowledge**: Classification, association, clustering, etc.
- **Background knowledge**: Concept hierarchies, constraints
- **Interestingness measures**: Support, confidence, thresholds
- **Visualization**: How to present results (rules, tables, charts)

DMQL Example:

```
use database Sales_DB
mine classification as Classify_Customers
from Customer_View
in relevance to age, income, region, buying_freq
overrule buying_freq = "High"
display as rules
```

Unit 4: Data Cube Technology

4.1 Data Cube Fundamentals

- **Data Cube**: n-dimensional representation of data for OLAP — metaphor for multidimensional storage
- **Cuboid**: A particular subset of dimensions forming a group-by. Each cuboid holds aggregated data at a specific granularity.
- **Lattice**: Partial order of cuboids from base (most detailed) to apex (most aggregated). Number of cuboids = 2^d for d dimensions.

Types of Cells in a Cube:

Cell Type	Description	Example
Base cell	Most detailed — all dimensions specified	(Q1, KTM, Laptop)
Aggregate cell	Some dimensions rolled up	(Year, KTM, Laptop) — time aggregated
M-dimensional cell	Exactly m non-* dimensions	2D cell: (Q1, KTM, *)

4.2 Cube Materialization

Cuboid Lattice: Base cuboid (most detailed) ↔ Apex cuboid (most generalized)

Type	Description
Full Cube	All cuboids precomputed — exponential $O(2^n)$
Iceberg Cube	Only cells above min threshold
Closed Cube	Only closed cells (no descendant with same measure)
Shell Cube	Precompute only small-dim cuboids, rest on-the-fly

Total cuboids with dimensions = d: 2^d (binary lattice)

If each dimension has L_i levels: Total cuboids = $\prod(L_i + 1)$

Example: 5 dimensions, each with 3 levels → $(3+1)^5 = 4^5 = 1024$ cuboids. Base cuboid = most detailed, Apex cuboid = total aggregate.

4.3 General Strategies for Cube Computation

Strategy	Description
Multiway Array Aggregation	Simultaneously aggregate multiple cuboids in one scan using multi-dimensional array chunks. Minimizes disk I/O by computing parent cuboids from smallest child.
BUC (Bottom-Up Computation)	Starts from apex, partitions data recursively. Efficient for iceberg cubes with high skew. Early pruning using min_sup.
Star Cubing	Integrates top-down (BUC) and bottom-up (Multiway). Uses shared dimensions for compression. Good for sparse and dense data.
High-dimensional OLAP	For very high dimensions — uses shell fragments, computes only necessary cuboids on demand.

Example — Multiway: For 3D cube (A,B,C), compute group-bys in order: ABC→AB→AC→BC→A→B→C→all. Cache smallest child cuboid to compute parent.

4.4 Optimization Strategies (with Examples)

1. **Sort, hash, group**: Sort dimension attributes to identify same groups faster. *Eg: Sort by (City, Item) before aggregating sales.*

2. **Compute from smallest child**: Aggregate from smallest child cuboid to minimize I/O. *Eg: Compute (City, Item) from (City, Item, Time) instead of base table.*
3. **Cache reuse**: Compute higher aggregates from cached lower-level results. *Eg: Once (City, Item) computed, cache it to compute (City) and (Item).*
4. **Apriori pruning**: For iceberg cubes, prune cells below threshold. *Eg: If min_sup=100, skip any cell with count < 100.*

4.5 Data Cube vs Attribute-Oriented Induction (AOI)

Aspect	Data Cube Approach	AOI
Approach	Off-line, precomputed	On-line, query-driven
Computation	Materializes all cuboids ahead of time	Generalizes on-the-fly per query
Storage	Large space (exponential)	Minimal — no pre-storage
Speed	Fast query response	Slower — processes at query time
Best for	Repeated queries, known patterns	Ad-hoc exploration, dynamic queries

4.6 Attribute-Oriented Induction (AOI) Steps

1. **Data Collection**: Gather task-relevant data from database
2. **Attribute Relevance Analysis**: Remove irrelevant/weak attributes
3. **Apply Generalization** using two rules:
   Attribute Removal: Remove attribute if it has too many distinct values AND no generalization operator exists (e.g., student ID)
   Attribute Generalization: Map attribute up its concept hierarchy (e.g., age: 25 → "young")
4. **Synchronous Generalization**: Generalize all attributes together to maintain consistency
5. **Presentation**: Show results as generalized relations, cross-tabs, rules, or graphs

4.7 Mining Class Comparisons

Compare target class vs contrasting classes

Steps: Data Collection → Dimension Relevance Analysis → Synchronous Generalization → Presentation (tables, graphs, rules)

Unit 5: Mining Frequent Patterns

5.1 Basic Concepts

Frequent Pattern: A pattern that appears frequently in a dataset.

Types of Frequent Patterns:

Pattern	Description	Example
Frequent Itemset	Set of items appearing together	{milk, bread} — in market basket
Frequent Subsequence	Ordered sequence of events	{buy phone → buy case} — sequential purchases
Frequent Substructure	Graph/tree/structural pattern	Benzene ring in chemical compounds

Market Basket Analysis: Analyzes customer transactions to find items frequently bought together. Helps retailers with product placement, promotions, cross-selling, and inventory management. Example: {milk, bread} → customers who buy milk often buy bread.

Term	Definition	Formula
Frequent Itemset	Set of items with count ≥ min_sup	count ≥ min_sup
Closed Itemset	No proper superset with same support	—
Maximal Frequent Itemset	No proper superset is frequent	—
Support (A⇒B)	% of transactions containing A∪B	$P(A \cup B)$ = count(A∪B) / n
Confidence (A⇒B)	% of A-transactions also containing B	$P(B \mid A) = P(A \cup B) / P(A)$
Lift	Correlation measure	$P(A \cup B) / (P(A)P(B))$

- *Lift* > 1 → positive correlation
- *Lift* < 1 → negative correlation
- *Lift* = 1 → independent

5.2 Association Rules

Association Rule: An implication of the form X ⇒ Y where X, Y are itemsets and X∩Y = ∅.

Types of Association Rules:

Type	Description	Example
Single dimensional	Single predicate repeated	buys(X,"camera") ⇒ buys(X,"printer")
Multidimensional	≥2 predicates	age(X,"young") ∧ income(X,"high") ⇒ buys(X,"SUV")
Multilevel	Mining at multiple concept hierarchy levels	milk ⇒ bread (low), 2%_milk ⇒ wheat_bread (high)
Quantitative	Numeric attributes with discretization	age: 20..30 ⇒ buys: "laptop"

5.3 Apriori Algorithm (Step-by-Step)

Apriori Property: Any subset of a frequent itemset must be frequent.

Steps for Numericals:

1. **Find L_1**: Scan DB, count each item, keep those with count ≥ min_sup
2. **Generate C_2**: Pair up frequent items from L_1 → all possible 2-item combinations

3. **Prune C_2**: Drop any pair whose individual item wasn't frequent in L_1
4. **Find L_2**: Scan DB, count the pairs in C_2, keep those ≥ min_sup
5. **Repeat** for k=3,4,...: Combine frequent k−1 itemsets into k-item candidates → prune → count → keep frequent ones
6. **Stop** when no more candidates can be formed
7. **Generate Rules**: From each frequent itemset, extract rules A ⇒ (L−A) where confidence ≥ min_conf

```
L₁ → C₂ → L₂ → C₃ → L₃ → ...
```

- **Generate Candidates**: Combine frequent k−1 itemsets to form k-item candidates
- **Prune**: Remove candidates that contain an infrequent subset

Limitations:

- Huge candidate generation (C_2 grows exponentially)
- Repeated DB scans (one per level)
- Slow for dense/long patterns

Example: DB with 9 transactions, min_sup = 2 - **Step 1** (Find L_1): Count each item → L_1 = {I1:6, I2:7, I3:6, I4:2, I5:2} - **Step 2** (Generate C_2): Pair up frequent items → C_2 = {I1I2, I1I3, I1I4, I1I5, I2I3, I2I4, I2I5, I3I4, I3I5, I4I5} - **Step 3** (Count → L_2): Scan, count pairs → L_2 = {I1I2:4, I1I3:4, I1I5:2, I2I3:4, I2I4:2, I2I5:2} - **Step 4** (Generate C_3): Combine L_2 items into triples → C_3 = {I1I2I3, I1I2I5, I1I3I5, I2I3I4, I2I3I5} → after prune: {I1I2I3, I1I2I5} - **Step 5** (Count → L_3): Scan, count triples → L_3 = {I1I2I3:2, I1I2I5:2} - **Step 6** (Stop): C_4 empty → done - **Step 7** (Rules): I1I2⇒I3 (conf=2/4=50%), I1I3⇒I2 (2/4=50%), I2I3⇒I1 (2/4=50%), I1I2⇒I5 (2/4=50%)

5.4 Improving Apriori Efficiency

Method	Description
Hash-based	Reduce C_k size using hash table bucketing
Transaction Reduction	Remove transactions without frequent k-itemsets
Partitioning	Mine partitions in memory, then merge
Sampling	Mine random sample (trade accuracy for speed)

5.6 FP-Growth Algorithm

- **No candidate generation** — uses FP-tree (divide & conquer)

Steps:

1. Scan DB, find frequent 1-itemsets, sort descending
2. Build FP-tree: root→null, insert transactions sorted, share prefixes
3. Mining: Bottom-up from leaves → conditional pattern base → conditional FP-tree

Advantage: Faster than Apriori for dense datasets

5.7 Generating Association Rules

For each frequent itemset L, generate non-empty subsets:

- For each subset A: rule A ⇒ (L - A)
- Keep rules where confidence ≥ min_conf

5.8 From Association to Correlation (Lift)

$Lift(A \Rightarrow B) = P(A \cup B) / (P(A) \times P(B))$

Example: 100 transactions, 40 contain milk, 30 contain bread, 20 contain both. P(milk)=0.4, P(bread)=0.3, P(both)=0.2. Lift = 0.2/(0.4×0.3) = 0.2/0.12 ≈ 1.67 > 1 → positive correlation (milk ↑ bread ↑)

Unit 6: Classification and Prediction

6.1 Learning and Testing of Classification

Classification: Predict categorical label (supervised) — e.g., spam or not spam **Prediction/Regression**: Predict continuous value — e.g., house price

Aspect	Classification	Prediction
Output	Discrete class labels	Continuous numeric value
Example	Yes/No, Cat/Dog, Spam/Ham	Price, temperature, sales
Metrics	Accuracy, Precision, Recall, F1	MSE, RMSE, MAE, R^2
Algorithms	DT, NB, SVM, KNN	Linear regression, Neural nets

Learning Step: Build model from training data (known labels) **Testing Step**: Evaluate accuracy on test data (unseen)

6.2 Classification by Decision Tree Induction

Top-down, greedy approach — recursively partitions data based on best attribute.

Attribute Selection Measures

ID3 — Information Gain

Entropy (Before Splitting): $E(D) = -\sum_{i=1}^{m} p_i \log_2 p_i$

Entropy (After Splitting on attribute A): $E_A(D) = \sum_{j=1}^{v} (|D_j| / |D|) \times E(D_j)$

Information Gain: $IG(A) = E(D) - E_A(D)$

Select attribute with highest IG as splitting criterion.

- Entropy = 0 when all samples same class (pure)
- Entropy = 1 when equally split (binary, max uncertainty)
- ID3 favors multi-valued attributes → use Gain Ratio (C4.5) to avoid bias

Example: 14 samples, 9 Yes, 5 No. Entropy = $-(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.940$. If Outlook splits into Sunny(3Y,2N), Overcast(4Y,0N), Rain(2Y,3N): E_Outlook = (5/14)×0.971 + (4/14)×0 + (5/14)×0.971 = 0.694. IG = 0.940 − 0.694 = 0.246

6.3 Naïve Bayesian Classification

Bayes' Theorem: $P(H \mid X) = P(X \mid H) \times P(H) / P(X)$

Naïve Bayes (Posterior for each class): $P(C_i \mid X) = P(C_i) \times \prod_{k=1}^{n} P(x_k \mid C_i)$

> **For exam**: Compute $P(C_i|X)$ for each class, pick the highest. Denominator $P(X)$ same for all classes — compare numerators only. Naïve Bayes assumes conditional independence — rarely true but works well. Use Laplace smoothing (+1 to each count) if any $P(x_k|C_i) = 0$.

Laplace Smoothing: $P(x_k|C_i) = (\text{count}(x_k, C_i) + 1) / (\text{count}(C_i) + n)$ where n = number of attribute values

Example: Class: Play=Yes(9), No(5). Outlook=Sunny: Yes=3, No=2. P(Sunny|Yes)=3/9=0.33, P(Sunny|No)=2/5=0.4. Laplace: P(S|Yes)=(3+1)/(9+3)=4/12=0.33, P(S|No)=(2+1)/(5+3)=3/8=0.375

6.4 Classification by Backpropagation

Definition: A neural network learning algorithm that adjusts weights by propagating error backwards from output to input. Uses Multilayer Perceptron (MLP): Input → Hidden → Output layers.

Steps:

1. **Initialize**: Set weights w_{ij} and biases b_j to small random values. Set learning rate η (e.g., 0.5).
2. **Forward Pass**: Compute hidden and output activations
   $I_j = \sum_i x_i w_{ij} + b_j$
   $O_j = 1 / (1 + e^{-I_j})$ (sigmoid activation)
3. **Backward Pass (Error Propagation)**:
   Output layer error: $\delta_j = O_j (1 - O_j) (T_j - O_j)$
   Hidden layer error: $\delta_j = O_j (1 - O_j) \sum_k \delta_k w_{jk}$
4. **Weight & Bias Update**:
   $w_{ij}^{new} = w_{ij}^{old} + \eta \times \delta_j \times O_i$
   $b_j^{new} = b_j^{old} + \eta \times \delta_j$

Repeat steps 2-4 until error converges or max epochs reached.

6.5 Rule-Based Classification

- **IF condition THEN class** (antecedent ANDed → consequent)

Coverage: $|D_{covers}| / |D|$ — proportion of all instances in DB that trigger the rule

Accuracy: $|D_{correct}| / |D_{covers}|$ — proportion of covered instances that are correctly classified

Example: If rule covers 80 out of 100 instances, and 60 of those are correctly classified: Coverage = 80/100 = 80%, Accuracy = 60/80 = 75%

- **Conflict Resolution**: Size ordering (more conditions = higher priority), Rule ordering (class-based or rule-based priority list)
- **Extraction from DT**: Each root→leaf path = one rule
- **Rule Simplification**: Prune conditions that don't improve accuracy

6.6 Support Vector Machine (SVM)

- Finds **maximum-margin hyperplane (MMH)** to separate classes
- **Support Vectors**: Data points closest to hyperplane (determine margin)
- **Linear SVM**: For linearly separable data
- **Non-linear SVM**: Uses kernel trick (RBF, polynomial, sigmoid)
- **Advantages**: Higher speed, good with limited samples

6.7 Performance Measures

Metric	Formula	Example (TP=50, TN=30, FP=10, FN=10)
Accuracy	(TP + TN) / (TP + TN + FP + FN)	(50+30)/100 = 80%
Precision	TP / (TP + FP)	50/60 ≈ 83.3%
Recall (Sensitivity)	TP / (TP + FN)	50/60 ≈ 83.3%
F1-Score	2 × Precision × Recall / (Precision + Recall)	2×0.833×0.833/(0.833+0.833) ≈ 83.3%

Metric	Formula	Example (TP=50, TN=30, FP=10, FN=10)
Specificity	TN / (TN + FP)	30/40 = 75%

- TP = Correct positive, TN = Correct negative
- FP = Type I error (false alarm), FN = Type II error (miss)
- For imbalanced data, use precision/recall/F1, not accuracy

Example: 100 samples, actual: 60 positive, 40 negative. Model predicts: TP=50, TN=30, FP=10, FN=10. Accuracy = 80/100=80%. Precision = 50/60≈83%. Recall = 50/60≈83%. F1 = 2×0.83×0.83/1.66≈0.83

6.8 Issues in Classification

- **Missing attribute values**: Handle via ignoring, filling with mean/mode, or predicting
- **Noisy data**: Mislabeled training examples degrade accuracy
- **High dimensionality**: Curse of dimensionality — too many features relative to samples
- **Imbalanced data**: Uneven class distribution — use precision/recall/F1 instead of accuracy
- **Overfitting**: Model memorizes noise instead of learning patterns
- **Underfitting**: Model too simple to capture data structure
- **Irrelevant features**: Degrade model performance — use feature selection

6.9 Model Evaluation

Overfitting vs Underfitting

Aspect	Overfitting	Underfitting
Model complexity	Too complex (captures noise)	Too simple (misses patterns)
Training accuracy	Very high (near 100%)	Low
Test/generalization	Poor — high variance	Poor — high bias
Cause	Too many features, deep trees, no pruning	Insufficient features, shallow trees
Fix	Prune, regularize, more data, reduce features	Add features, increase model complexity

k-Fold Cross Validation

- Split data into k equal folds (typically k = 5 or 10)
- For each fold i: train on k−1 folds, test on fold i
- Repeat k times — each fold used once as test
- Final accuracy = average of all k runs
- **Advantages**: All data used for both training and testing, reduces variance of estimate, robust to data partitioning

McNemar's Test (Comparing Two Classifiers)

Used to test if two classifiers have significantly different performance.

Steps:

1. **Construct contingency table** from test set predictions:

	Classifier B: Correct	Classifier B: Wrong
Classifier A: Correct	n_{00} (both correct)	n_{01} (A correct, B wrong)
Classifier A: Wrong	n_{10} (A wrong, B correct)	n_{11} (both wrong)

1. **Hypothesis**:
   H_0: Both classifiers have same error rate ($n_{01} = n_{10}$)
   H_1: Classifiers differ significantly

2. **Test statistic**: $\chi^2 = (n_{01} - n_{10})^2 / (n_{01} + n_{10})$

3. **Decision**: If $\chi^2 > 3.84$ (critical at $\alpha = 0.05$, df = 1), reject H_0 — classifiers differ significantly.

Unit 7: Cluster Analysis

7.1 Types of Data in Cluster Analysis

Cluster analysis works with various data types:

- **Interval-scaled**: Continuous measurements (height, weight, temperature)
- **Binary**: Presence/absence (0/1) — symmetric vs asymmetric
- **Nominal**: Categories (color, country)
- **Ordinal**: Ordered categories (small, medium, large)
- **Mixed**: Combination of multiple types

7.2 Similarity and Dissimilarity Between Objects

- **Similarity**: Higher value = more alike (ranges 0 to 1)
- **Dissimilarity (Distance)**: Lower value = more alike
- Similarity = 1 − Dissimilarity (if normalized)

Distance Measures:

Euclidean Distance: $d = \sqrt{((x_2-x_1)^2 + (y_2-y_1)^2)}$

Manhattan Distance: $d = |x_2-x_1| + |y_2-y_1|$

Minkowski Distance (generalized): $d = (|x_2-x_1|^p + |y_2-y_1|^p)^{1/p}$

- $p = 1 \rightarrow$ Manhattan (L_1)
- $p = 2 \rightarrow$ Euclidean (L_2)

For binary data: Simple matching coefficient (SMC) = $(f_{11} + f_{00}) / (f_{11} + f_{10} + f_{01} + f_{00})$ **Jaccard coefficient** (for asymmetric binary): $(f_{11}) / (f_{11} + f_{10} + f_{01})$

7.3 Comparison of Clustering Methods

Aspect	Partitioning (K-Means)	Hierarchical (Agglomerative)	Density-Based (DBSCAN)
Shape	Spherical clusters only	Any shape (based on linkage)	Arbitrary shapes
Number of clusters	Must specify k	Cut dendrogram at level	Auto — eps & minPts determine
Outliers	Sensitive (mean pulls)	Sensitive (single link)	Robust — marks as noise
Scalability	O(n·k·i) — good	$O(n^2)$ or $O(n^3)$ — poor	$O(n^2)$ — moderate
Deterministic	No (depends on init)	Yes (deterministic merge)	Yes (order doesn't matter)
Assumes	Spherical, equal size	No assumption	Dense regions separated by sparse

7.4 Partitioning Methods

K-Means Algorithm

Algorithm:

1. Choose *k* and randomly select *k* initial centroids from data points
2. **Assign**: Compute distance of each point to each centroid → assign to nearest
3. **Update**: Recompute centroids = mean of all points in each cluster
4. Repeat steps 2–3 until centroids don't change (convergence)

Step-by-Step Example: Points: A(2,10), B(2,5), C(8,4), D(5,8), E(7,5), F(6,4); k=2

Init: Random centroids c_1=(2,5), c_2=(6,4)

Point	Dist to c_1	Dist to c_2	Assign
A(2,10)	√(0+25)=5.00	√(16+36)=7.21	c_1
B(2,5)	√(0+0)=0.00	√(16+1)=4.12	c_1
C(8,4)	√(36+1)=6.08	√(4+0)=2.00	c_2
D(5,8)	√(9+9)=4.24	√(1+16)=4.12	c_2
E(7,5)	√(25+0)=5.00	√(1+1)=1.41	c_2
F(6,4)	√(16+1)=4.12	√(0+0)=0.00	c_2

C_1 = {A,B}; **C_2** = {C,D,E,F}

Update: c_1 = ((2+2)/2, (10+5)/2) = (2, 7.5); c_2 = ((8+5+7+6)/4, (4+8+5+4)/4) = (6.5, 5.25)

Iter 2: Reassign with new centroids → C_1 = {A,B,D}, C_2 = {C,E,F}

Update: c_1 = ((2+2+5)/3, (10+5+8)/3) = (3, 7.67); c_2 = ((8+7+6)/3, (4+5+4)/3) = (7, 4.33)

Iter 3: Same assignment → **Converged!** Final clusters: {A,B,D}, {C,E,F}

K-Means++ (Better Initialization)

Algorithm:

1. Choose first centroid randomly from data points
2. For each point, compute distance d to nearest already-chosen centroid
3. Select next centroid from remaining points with probability $\propto d^2$ (farther points more likely)
4. Repeat steps 2–3 until k centroids chosen
5. Run standard K-Means with these initial centroids

Mini-Batch K-Means

Algorithm:

1. Choose k and batch size b
2. Initialize centroids randomly (or via K-Means++)
3. **Sample**: Pick random batch of b points from dataset
4. **Assign**: Assign each batch point to nearest centroid
5. **Update**: Update centroids using only batch points (learning rate controls influence)
6. Repeat steps 3–5 until convergence (across batches)

K-Medoids (PAM — Partitioning Around Medoids)

Algorithm:

1. Choose k and randomly select k medoids (actual data points as centers)
2. **Assign**: Assign each point to nearest medoid
3. **Swap**: For each medoid m and non-medoid p:
   Swap $m \leftrightarrow p$ and compute total cost = Σ distance(point, nearest medoid)
   Keep swap if cost decreases
4. Repeat steps 2–3 until no swap reduces cost

Advantage: Robust to outliers (actual data points as centers, not means)

7.5 Hierarchical Methods

Type	Description
Agglomerative (Bottom-up)	Start with each point as cluster; merge closest pairs
Divisive (Top-down)	Start with one cluster; recursively split

- Results shown as **dendrogram**
- Distance matrix determines merging

Agglomerative Clustering Algorithm

Algorithm:

1. Let each data point be its own cluster
2. Compute distance matrix between all pairs of clusters
3. **Repeat** until one cluster remains:
   a. Merge the two closest clusters (minimum distance)
   b. Update distance matrix using linkage criterion
4. Cut dendrogram at desired level to get *k* clusters

Linkage Criteria:

- **Single link**: min distance between any point in C_i and C_j
- **Complete link**: max distance
- **Average link**: average of all pairwise distances
- **Ward's**: minimize increase in SSE after merging

7.6 Density-Based Methods

DBSCAN Algorithm

Algorithm:

1. Choose ε (radius) and minPts (minimum points for dense region)
2. For each point, count neighbors within ε distance
3. Label as:
   Core point: ≥ minPts neighbors within ε
   Border point: < minPts but within ε of a core point
   Noise: neither core nor border
4. Form clusters: connect core points within ε of each other; add border points to their core's cluster
5. Output clusters and noise points

Example: Points {(2,10),(2,5),(8,4),(5,8),(7,5),(6,4),(1,2),(4,9)}, ε=2, minPts=2

- Core: C(8,4), D(5,8), E(7,5), F(6,4), H(4,9)
- Noise: A(2,10), B(2,5), G(1,2)
- Clusters: {C,E,F} and {D,H}

7.7 Outlier Analysis

- **Outlier**: Data point deviating from overall pattern
- Common causes: data entry errors, measurement errors, natural novelties
- Detected via: clustering (noise points), statistical tests, distance-based methods
- Important in: fraud detection, medical anomalies

Unit 8: Graph Mining and Social Network Analysis

8.1 Graph Mining

- Extracting frequent subgraphs/patterns from graph data

Why Graph Mining?

- Many data types are naturally represented as graphs (social networks, chemical compounds, web links, biological networks)
- Traditional pattern mining ignores structural relationships between entities
- Graph mining captures complex relationships, paths, and substructures
- **Applications**: chemical compounds, protein structures, social networks, web, fraud detection

8.2 Graph Mining Algorithms

Beam Search

- Heuristic search: retains only β most promising nodes at each level (beam width)
- Memory-efficient alternative to BFS/DFS/A*

```
Open = {initial state}
while Open not empty:
  n = best node from Open
  if n is goal → return path
  successors = expand(n)
  Open = Open ∪ successors
  if |Open| > β: keep best β, drop rest
```

Inductive Logic Programming (ILP)

- Intersection of ML and Logic Programming
- Derives rules from positive/negative examples + background knowledge
- **Induction**: specific → general (rules from facts)
- **Deduction**: general → specific (facts from rules)

PageRank Algorithm

- Ranks web pages by importance based on link structure
- **Idea**: A page is important if many important pages link to it
- **PR**(u) = (1 − d) + d × $\Sigma_{v \in B(u)}$ PR(v) / N_v
- d = damping factor (typically 0.85)
- $B(u)$ = pages linking to u
- N_v = out-degree of page v

Algorithm:

1. Initialize all pages with PR = 1 / N
2. **Repeat** until convergence:
   For each page u, compute PR via formula
   Sum of all PR values = N (conservation)
3. Output ranked list of pages by PR
4. **Teleportation**: (1−d) factor handles dangling links (pages with no out-links)
5. **Applications**: Web search ranking, citation analysis, recommendation systems

8.3 Social Network Analysis

- Study of social structures via nodes (actors) and edges (relationships)
- **Challenges**: large scale, noisy data, dynamic data

8.4 Link Mining Tasks

Task	Description
Link-based classification	Predict node label based on links
Object type prediction	Predict type of object
Link type prediction	Predict type of relationship
Link existence	Predict if link exists
Link cardinality	Predict number of links
Object reconciliation	Match objects across sources
Group detection	Detect communities

8.5 Friends of Friends & Degree Assortativity

- **FoF**: C is friend of A's friend B → indirect connection (recommendations)
- **Degree Assortativity**: Tendency of nodes to connect to similar-degree nodes
- Positive coefficient = assortative (social networks)
- Negative coefficient = disassortative
- Measured by Pearson correlation of degrees

8.6 Signed Networks

- Edges can be positive (friend/like) or negative (enemy/dislike)

Balance Theory:

- Balanced triples: 3 positives OR 1 positive + 2 negatives
- "Friend of my friend is my friend"
- "Enemy of my friend is my enemy"

Status Theory:

- Positive link → higher status
- Negative link → lower status

Conflict: Balance and status can give opposite predictions (A→B→C→A triangle)

Example: A→B (+), B→C (+), C→A (−). Balance: 2 positives + 1 negative → balanced (one negative edge). Status: A>B, B>C → A>C, but C→A is negative → status says A<C. Conflict! Balance says OK, status says contradictory.

8.7 Trust Propagation

Atomic Propagation:

- Direct (i→j→k ⇒ i trusts k)
- Co-citation, Transpose trust, Trust coupling

Propagation of Distrust:

- Trust-only, One-step distrust, Propagated distrust (B = T − D)

Iterative Propagation:

- Eigenvalue propagation, Weighted linear combination

8.8 Predicting Positive and Negative Links

- Predict the sign of an unknown edge in a signed network based on known edges
- **Balance theory prediction**: A triangle with an even number of negative edges is balanced → predict positive

- **Status theory prediction**: If A has higher status than B and B has higher status than C, then A > C → predict positive
- **Machine learning approach**: Use features like degree, triad counts, node attributes with classifiers (logistic regression, SVM)

Unit 9: Mining Spatial, Multimedia, Text, and Web Data

9.1 Spatial Data Mining

- Extract knowledge from spatial databases (maps, remote sensing)
- **Spatial Data Cube**:
- Nonspatial dimension
- Spatial-to-nonspatial dimension
- Spatial-to-spatial dimension
- **Measures**: Numerical (revenue) or Spatial (pointers to objects)
- **Applications**: GIS, geo-marketing, remote sensing

Mining Spatial Association:

- Find association rules involving spatial predicates (e.g., nearby, contains, adjacent)
- Example: "Is close_to(City, Highway) ∧ Is_near(City, Airport) → Is_growing(City)"
- **Approach**: Progressive refinement (coarse filter → fine filter) using spatial indexes
- **Spatial predicates**: topological (touches, contains), distance (near, far), direction (north, east)

9.2 Multimedia Data Mining

- Integrates image processing, computer vision, data mining
- **Description-based Retrieval**: Keywords, captions, metadata
- **Content-based Retrieval**: Color histogram, texture, shape, layout
- **Association Types**:
- Image↔non-image
- Image↔image (non-spatial)
- Image↔image (spatial relationships)

9.3 Text Mining & NLP

- Derive high-quality info from text documents
- **Tasks**: text classification, clustering, sentiment analysis, summarization
- **Information Extraction (IE)**: Extracts structured data from unstructured text
- **NLP**: Natural Language Understanding (NLU) + Natural Language Generation (NLG)

9.4 Web Mining

Category	Description
Web Content Mining	Extract info from web pages — text, multimedia
Web Structure Mining	Analyze hyperlink structure between pages
Web Usage Mining	Mine server logs for browsing patterns